



MAX PLANCK INSTITUTE
FOR DYNAMICS OF COMPLEX
TECHNICAL SYSTEMS
MAGDEBURG



COMPUTATIONAL METHODS IN
SYSTEMS AND CONTROL THEORY

Mixed-precision iterative refinement for low-rank Lyapunov equations

Xiaobo Liu

Max Planck Institute for Dynamics of Complex Technical Systems,
Germany

**Scientific Computation Seminar,
University of Leeds, August 14, 2025**

Joint work with Peter Benner



1. Background

- Low-rank Lyapunov equations
- Floating-point arithmetics
- Use of low and mixed precisions

2. Mixed-precision iterative refinement for linear systems

- For $Ax = b$
- For low-rank Lyapunov equation
- Rounding error analysis

3. Sign function Newton iteration as solver

- Sign function Newton iteration
- Cost analysis

4. Numerical experiments



Low-rank (algebraic) Lyapunov equations

$$AX + XA^T + W = 0, \quad W = LL^T \text{ or } LSL^T, \quad L \in \mathbb{R}^{n \times m}, \quad m \ll n.$$

App. in control theory, system balancing, and model reduction, etc.

- $n \sim 10^4$ or beyond
- $\operatorname{Re} \lambda_i(A) < 0$, W is PSD
- X is PSD & has small numer. rank:

$$\|X - X_\varepsilon\| \leq u\|X\|, \quad \operatorname{rank}(X_\varepsilon) \ll n$$

$\leadsto X = ZZ^T$, factor $Z \approx$ low-rank matrix (Komaroff, '88).



Low-rank (algebraic) Lyapunov equations

$$AX + XA^T + W = 0, \quad W = LL^T \text{ or } LSL^T, \quad L \in \mathbb{R}^{n \times m}, \quad m \ll n.$$

App. in control theory, system balancing, and model reduction, etc.

- $n \sim 10^4$ or beyond
- $\operatorname{Re} \lambda_i(A) < 0$, W is PSD
- X is PSD & has small numer. rank:

$$\|X - X_\varepsilon\| \leq u\|X\|, \quad \operatorname{rank}(X_\varepsilon) \ll n$$

$\leadsto X = ZZ^T$, factor $Z \approx$ low-rank matrix (Komaroff, '88).



Low-rank (algebraic) Lyapunov equations

$$AX + XA^T + W = 0, \quad W = LL^T \text{ or } LSL^T, \quad L \in \mathbb{R}^{n \times m}, \quad m \ll n.$$

App. in control theory, system balancing, and model reduction, etc.

- $n \sim 10^4$ or beyond
- $\operatorname{Re} \lambda_i(A) < 0$, W is PSD
- X is PSD & has small numer. rank:

$$\|X - X_\varepsilon\| \leq u\|X\|, \quad \operatorname{rank}(X_\varepsilon) \ll n$$

$\rightsquigarrow X = ZZ^T$, factor $Z \approx$ low-rank matrix (Komaroff, '88).



Table: Parameters of four floating point formats.

Type	Signif. bits (t)	Exp. bits	Range	$u = 2^{-t}$
bf16	8	8	$10^{\pm 38}$	3.9×10^{-3}
fp16	11	5	$10^{\pm 5}$	4.9×10^{-4}
fp32	24	8	$10^{\pm 38}$	6.0×10^{-8}
fp64	53	11	$10^{\pm 308}$	1.1×10^{-16}

- Lower precision $\rightsquigarrow$ faster flops, less comm., lower energy consumption.



Table: Parameters of four floating point formats.

Type	Signif. bits (t)	Exp. bits	Range	$u = 2^{-t}$
bf16	8	8	$10^{\pm 38}$	3.9×10^{-3}
fp16	11	5	$10^{\pm 5}$	4.9×10^{-4}
fp32	24	8	$10^{\pm 38}$	6.0×10^{-8}
fp64	53	11	$10^{\pm 308}$	1.1×10^{-16}

- Lower precision $\rightsquigarrow$ faster flops, less comm., lower energy consumption.



When to use low precision?

- Low accuracy is needed
- Other sources of errors:
model reduction, approximation (e.g., low rank, discretization)
- Self-correction mechanism (e.g., iterative refinement) ✓

⇒ **Mixed precision algorithm:** select **diff. prec.** for **diff. parts** of comput.
to **improve performance** while **guarantee accuracy and stability**.



When to use low precision?

- Low accuracy is needed
- Other sources of errors:
model reduction, approximation (e.g., low rank, discretization)
- Self-correction mechanism (e.g., iterative refinement) ✓

⇒ **Mixed precision algorithm:** select diff. prec. for diff. parts of comput.
to improve performance while guarantee accuracy and stability.



When to use low precision?

- Low accuracy is needed
- Other sources of errors:
model reduction, approximation (e.g., low rank, discretization)
- Self-correction mechanism (e.g., iterative refinement) ✓

⇒ **Mixed precision algorithm**: select **diff. prec.** for **diff. parts** of comput.
to **improve performance** while **guarantee accuracy and stability**.



1. Background

- Low-rank Lyapunov equations
- Floating-point arithmetics
- Use of low and mixed precisions

2. Mixed-precision iterative refinement for linear systems

- For $Ax = b$
- For low-rank Lyapunov equation
- Rounding error analysis

3. Sign function Newton iteration as solver

- Sign function Newton iteration
- Cost analysis

4. Numerical experiments



For $Ax = b$, use three prec. $u_r \leq u \leq u_s$: (Carson & Higham, '18)

- u , working precision
- u_s , solving correction equation
- u_r , residual computation

-
- 1 Solve $Ax = b$ in u_s and store x at u
 - 2 **repeat**
 - 3 Comput. $r = b - Ax$ at u_r and round r to u_s
 - 4 Solve $Ad = r$ at u_s and store d at u
 - 5 $x \leftarrow x + d$ at u .
 - 6 **until** cge .
-
- B'err of $O(u)$, subject to $\kappa(A)$



- Existing work: Solving $AX + XA^T + LL^T = 0$ in two-prec. iterative refinement (IR) (**Benner et al., '11**).
 - fp32-fp64 Implementation in hybrid CPU-GPU platforms
 - Chol-type** sol'n factor Z of $X = ZZ^T$
 - No rounding error analysis
- Our contributions:
 - Gen. to three-prec. IR framework
 - LDL^T-type** sol'n factor $X = ZYZ^T$
 - Mixed-precision error analysis for both types



- Existing work: Solving $AX + XA^T + LL^T = 0$ in two-prec. iterative refinement (IR) (**Benner et al., '11**).
 - fp32-fp64 Implementation in hybrid CPU–GPU platforms
 - **Chol-type** sol'n factor Z of $X = ZZ^T$
 - No rounding error analysis
- Our contributions:
 - Gen. to three-prec. IR framework
 - **LDL^T-type** sol'n factor $X = ZYZ^T$
 - Mixed-precision error analysis for both types



Comput. $X = ZYZ^T$ of $AX + XA^T + LSL^T = 0$.

- 1 Solve $AX + XA^T + LSL^T = 0$ in u_s and store Z and Y at u
 - 2 **repeat**
 - 3 Comput. $\mathcal{R}(Z, Y) := AZYZ^T + ZYZ^TA^T + LSL^T = L_\Delta S_\Delta L_\Delta^T$
 at u_r and round L_Δ and S_Δ to u_s
 - 4 Solve $AX_\Delta + X_\Delta A^T + L_\Delta S_\Delta L_\Delta^T = 0$ at u_s and store Z_Δ and Y_Δ
 (of $X_\Delta = Z_\Delta Y_\Delta Z_\Delta^T$) at u
 - 5 Find factors Z and Y of the *nearest* PSD matrix to $X \leftarrow X + X_\Delta$
 at u_c . ► *RHS not PSD & Y, Z obtained without forming $n \times n$ matrices*
 - 6 **until** *cge*.
-

- Res comput. & sol'n update, $O(n^2)$ in prec. higher than u_s



Comput. $X = ZYZ^T$ of $AX + XA^T + LSL^T = 0$.

- 1 Solve $AX + XA^T + LSL^T = 0$ in u_s and store Z and Y at u
 - 2 **repeat**
 - 3 Comput. $\mathcal{R}(Z, Y) := AZYZ^T + ZYZ^TA^T + LSL^T = L_\Delta S_\Delta L_\Delta^T$
 at u_r and round L_Δ and S_Δ to u_s
 - 4 Solve $AX_\Delta + X_\Delta A^T + L_\Delta S_\Delta L_\Delta^T = 0$ at u_s and store Z_Δ and Y_Δ
 (of $X_\Delta = Z_\Delta Y_\Delta Z_\Delta^T$) at u
 - 5 Find factors Z and Y of the *nearest* PSD matrix to $X \leftarrow X + X_\Delta$
 at u_c . ► *RHS not PSD & Y, Z obtained without forming $n \times n$ matrices*
 - 6 **until** *cge*.
-

- Res comput. & sol'n update, $O(n^2)$ in prec. higher than u_s



Comput. $X = ZYZ^T$ of $AX + XA^T + LSL^T = 0$.

- 1 Solve $AX + XA^T + LSL^T = 0$ in u_s and store Z and Y at u
 - 2 **repeat**
 - 3 Comput. $\mathcal{R}(Z, Y) := AZYZ^T + ZYZ^TA^T + LSL^T = L_\Delta S_\Delta L_\Delta^T$
 at u_r and round L_Δ and S_Δ to u_s
 - 4 Solve $AX_\Delta + X_\Delta A^T + L_\Delta S_\Delta L_\Delta^T = 0$ at u_s and store Z_Δ and Y_Δ
 (of $X_\Delta = Z_\Delta Y_\Delta Z_\Delta^T$) at u
 - 5 Find factors Z and Y of the *nearest* PSD matrix to $X \leftarrow X + X_\Delta$
 at u_c . ► *RHS not PSD & Y, Z obtained without forming $n \times n$ matrices*
 - 6 **until** cge.
-

- Res comput. & sol'n update, $O(n^2)$ in prec. higher than u_s



Comput. $X = ZYZ^T$ of $AX + XA^T + LSL^T = 0$.

- 1 Solve $AX + XA^T + LSL^T = 0$ in u_s and store Z and Y at u
 - 2 **repeat**
 - 3 Comput. $\mathcal{R}(Z, Y) := AZYZ^T + ZYZ^TA^T + LSL^T = L_\Delta S_\Delta L_\Delta^T$
 at u_r and round L_Δ and S_Δ to u_s
 - 4 Solve $AX_\Delta + X_\Delta A^T + L_\Delta S_\Delta L_\Delta^T = 0$ at u_s and store Z_Δ and Y_Δ
 (of $X_\Delta = Z_\Delta Y_\Delta Z_\Delta^T$) at u
 - 5 Find factors Z and Y of the *nearest* PSD matrix to $X \leftarrow X + X_\Delta$
 at u_c . ► *RHS not PSD & Y, Z obtained without forming $n \times n$ matrices*
 - 6 **until** cge.
-

- Res comput. & sol'n update, $O(n^2)$ in prec. higher than u_s



1. Rewrite $\mathcal{R}(Z) = AZYZ^T + ZYZ^T A^T + LSL^T$ as

$$\begin{aligned}\mathcal{R}(Z) &= \begin{bmatrix} Z & AZ & L \end{bmatrix} \begin{bmatrix} 0 & Y & 0 \\ Y & 0 & 0 \\ 0 & 0 & S \end{bmatrix} \begin{bmatrix} Z^T \\ Z^T A^T \\ L^T \end{bmatrix} \\ &=: FNF^T, \quad F \text{ tall and skinny (as } Z \text{ and } L \text{ are).}\end{aligned}$$

2. Comput. thin QR decom. $F = UT$, form $H = TNT^T$, and comput. spec. decom. $H = Q\Lambda Q^T$.

3. Trunc. e'vals $|\lambda_i| < \eta_r$, $\eta_r > 0$,

$$\mathcal{R}(Z) \approx UQ_t\Lambda_tQ_t^TU^T =: L_\Delta S_\Delta L_\Delta^T,$$

where

$$L_\Delta = UQ_t, \quad S_\Delta = \Lambda_t.$$



1. Rewrite $\mathcal{R}(Z) = AZYZ^T + ZYZ^T A^T + LSL^T$ as

$$\begin{aligned}\mathcal{R}(Z) &= \begin{bmatrix} Z & AZ & L \end{bmatrix} \begin{bmatrix} 0 & Y & 0 \\ Y & 0 & 0 \\ 0 & 0 & S \end{bmatrix} \begin{bmatrix} Z^T \\ Z^T A^T \\ L^T \end{bmatrix} \\ &=: FNF^T, \quad F \text{ tall and skinny (as } Z \text{ and } L \text{ are).}\end{aligned}$$

2. Comput. thin QR decom. $F = UT$, form $H = TNT^T$, and comput. spec. decom. $H = Q\Lambda Q^T$.

3. Trunc. e'vals $|\lambda_i| < \eta_r$, $\eta_r > 0$,

$$\mathcal{R}(Z) \approx UQ_t\Lambda_tQ_t^TU^T =: L_\Delta S_\Delta L_\Delta^T,$$

where

$$L_\Delta = UQ_t, \quad S_\Delta = \Lambda_t.$$



1. Rewrite $\mathcal{R}(Z) = AZYZ^T + ZYZ^T A^T + LSL^T$ as

$$\begin{aligned}\mathcal{R}(Z) &= \begin{bmatrix} Z & AZ & L \end{bmatrix} \begin{bmatrix} 0 & Y & 0 \\ Y & 0 & 0 \\ 0 & 0 & S \end{bmatrix} \begin{bmatrix} Z^T \\ Z^T A^T \\ L^T \end{bmatrix} \\ &=: FNF^T, \quad F \text{ tall and skinny (as } Z \text{ and } L \text{ are).}\end{aligned}$$

2. Comput. thin QR decom. $F = UT$, form $H = TNT^T$, and comput. spec. decom. $H = Q\Lambda Q^T$.

3. **Trunc.** e'vals $|\lambda_i| < \eta_r$, $\eta_r > 0$,

$$\mathcal{R}(Z) \approx UQ_t\Lambda_tQ_t^TU^T =: L_\Delta S_\Delta L_\Delta^T,$$

where

$$L_\Delta = UQ_t, \quad S_\Delta = \Lambda_t.$$



Update Z and Y with Z_Δ and Y_Δ .

1. Form

$$G := \begin{bmatrix} Z & Z_\Delta \end{bmatrix}, \quad G \text{ tall and skinny, and } \Upsilon = \begin{bmatrix} Y \\ Y_\Delta \end{bmatrix}.$$

2. Comput. *thin* QR decom. $G = V\Gamma$, form $K = \Gamma\Upsilon\Gamma^T$, and comput. spec. decom. $K = \Theta\Sigma\Theta^T$.

3. Trunc. e'vals $\sigma_i < \eta_s$, $\eta_s > 0$,

$$Z_{\text{new}} = V\Theta_t, \quad Y_{\text{new}} = \Sigma_t,$$

such that

$$X_{\text{new}} \approx V\Theta_t\Sigma_t\Theta_t^TV^T = Z_{\text{new}}Y_{\text{new}}Z_{\text{new}}^T.$$



Update Z and Y with Z_Δ and Y_Δ .

1. Form

$$G := \begin{bmatrix} Z & Z_\Delta \end{bmatrix}, \quad G \text{ tall and skinny, and } \Upsilon = \begin{bmatrix} Y \\ Y_\Delta \end{bmatrix}.$$

2. Comput. *thin* QR decom. $G = V\Gamma$, form $K = \Gamma\Upsilon\Gamma^T$, and comput. spec. decom. $K = \Theta\Sigma\Theta^T$.

3. Trunc. e'vals $\sigma_i < \eta_s$, $\eta_s > 0$,

$$Z_{\text{new}} = V\Theta_t, \quad Y_{\text{new}} = \Sigma_t,$$

such that

$$X_{\text{new}} \approx V\Theta_t\Sigma_t\Theta_t^TV^T = Z_{\text{new}}Y_{\text{new}}Z_{\text{new}}^T.$$



Update Z and Y with Z_Δ and Y_Δ .

1. Form

$$G := \begin{bmatrix} Z & Z_\Delta \end{bmatrix}, \quad G \text{ tall and skinny, and } \Upsilon = \begin{bmatrix} Y \\ Y_\Delta \end{bmatrix}.$$

2. Comput. *thin* QR decom. $G = V\Gamma$, form $K = \Gamma\Upsilon\Gamma^T$, and comput. spec. decom. $K = \Theta\Sigma\Theta^T$.

3. Trunc. e'vals $\sigma_i < \eta_s$, $\eta_s > 0$,

$$Z_{\text{new}} = V\Theta_t, \quad Y_{\text{new}} = \Sigma_t,$$

such that

$$X_{\text{new}} \approx V\Theta_t\Sigma_t\Theta_t^TV^T = Z_{\text{new}}Y_{\text{new}}Z_{\text{new}}^T.$$



1. Approx'n to full sol'n/incre/res computed **exactly** from factors:

$$\widehat{X} = \widehat{Z}\widehat{Y}\widehat{Z}^T, \quad \widehat{X}_\Delta = \widehat{Z}_\Delta\widehat{Y}_\Delta\widehat{Z}_\Delta^T, \quad \widehat{\mathcal{R}}(\widehat{Z}, \widehat{Y}) = \widehat{L}_\Delta\widehat{S}_\Delta\widehat{L}_\Delta^T.$$

- Why?

- Accuracy measured w.r.t. full sol'n/res
- $n \times n$ matrices never formed

2. Solver for $\mathcal{L}(X_\Delta) + \widehat{L}_\Delta\widehat{S}_\Delta\widehat{L}_\Delta^T = 0$ reasonably **stable**:

$$\|\mathcal{L}(\widehat{X}_\Delta) + \widehat{\mathcal{R}}(\widehat{Z}, \widehat{Y})\|_F \leq u_s (c_1 \|\mathcal{L}\|_F \|\widehat{X}_\Delta\|_F + c_2 \|\widehat{\mathcal{R}}(\widehat{Z}, \widehat{Y})\|_F),$$

- Implication: rel. res of $\mathcal{L}(X_i^\Delta) + \widehat{\mathcal{R}}(\widehat{Z}_i, \widehat{Y}_i) = 0 \sim$ at most $\max(c_1, c_2)u_s$



- Round-to-nearest, standard floating-point arithmetic model (**Higham, '02**):
 - **gemm**, comput., e.g., F (tall-and-skinny), H (small, square) (**Higham, '02**)
 - **thin QR**, of assembled res/sol'n factors F , G (Householder QR (**Higham, '02**))
 - **sym. spec. decom.**, of H : sym. QR (**Golub & Van Loan, '13**) or divide-and-conquer (**Gu & Eisenstat, '95**), (**Demmel, '97**) after tridiag
 - truncations of e'vals



Theorem

If $\psi := (c_1 \kappa_F(\mathcal{L}) + c_2)u_s$ is sufficiently less than 1, then the relative residual is reduced on the i th iteration by a factor approximately

$$\phi := \psi + 2\sqrt{m_i}\eta_r + c_3(2b_1b_2 + (4b_2 + 2)m_i^{3/2})nu_r$$

until an iterate pair $(\hat{Z}, \hat{Y})$ is produced, such that

$$\|\mathcal{R}(\hat{Z}, \hat{Y})\|_F \lesssim (2\sqrt{p_i}\eta_s + \eta_n + c_4(4b_3 + 2)p_i^{3/2}nu_c)\|\mathcal{L}\|_F\|\hat{Z}\hat{Y}\hat{Z}^T\|_F.$$

- ψ : problem conditioning and solver stability
- Dimensions $m_i, p_i \ll n$
- b_1, b_2, b_3 : stability of some gemm
- Tol: $\eta_r \ll 1$ for res factor trunc ($= 10^{-4}$ in (Benner et al., '11)),
 $\eta_s, \eta_n \sim nu$ for sol'n factor trunc

$\rightsquigarrow$ Res reduction if $\kappa_F(\mathcal{L})u_s$ sufficiently < 1 , limiting res $\sim nu$ if $u_c \lesssim u$.



Table: Condition number bounds and limiting residuals, given a *stable* solver.

u_s	$u_r = u_c = u$	Bound on $\kappa_F(\mathcal{L})$	Limiting residual
bf16	fp32	10^3	fp32
fp16		10^4	
fp32		10^8	
bf16	fp64	10^3	fp64
fp16		10^4	
fp32		10^8	
fp64		10^{16}	



1. Background

- Low-rank Lyapunov equations
- Floating-point arithmetics
- Use of low and mixed precisions

2. Mixed-precision iterative refinement for linear systems

- For $Ax = b$
- For low-rank Lyapunov equation
- Rounding error analysis

3. Sign function Newton iteration as solver

- Sign function Newton iteration
- Cost analysis

4. Numerical experiments



- Sign function Newton iteration as **solver** for computing $X = ZYZ^T$ of

$$AX + XA^T + W = 0, \quad W = LSL^T.$$

- Easy but robust implementation
 - Excellent parallelism
 - Rich in level-3 BLAS
- Different solvers also applicable—need to return the sol'n factors in the **Chol/LDL^T-type**.



- Derivation: with the block diag (**Roberts, '80**)

$$\begin{aligned} B &:= \begin{bmatrix} A & W \\ 0 & -A^T \end{bmatrix} = \begin{bmatrix} A & -AX - XA^T \\ 0 & -A^T \end{bmatrix} \\ &= \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix} \begin{bmatrix} A & \\ & -A^T \end{bmatrix} \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix}^{-1}, \quad B \in \mathbb{R}^{2n \times 2n}, \end{aligned}$$

we have

$$\text{sign}(B) = \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix} \begin{bmatrix} -I_n & \\ & I_n \end{bmatrix} \begin{bmatrix} I_n & -X \\ 0 & I_n \end{bmatrix} = \begin{bmatrix} -I_n & 2X \\ 0 & I_n \end{bmatrix}.$$



- Derivation: with the block diag (**Roberts, '80**)

$$\begin{aligned} B &:= \begin{bmatrix} A & W \\ 0 & -A^T \end{bmatrix} = \begin{bmatrix} A & -AX - XA^T \\ 0 & -A^T \end{bmatrix} \\ &= \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix} \begin{bmatrix} A & \\ & -A^T \end{bmatrix} \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix}^{-1}, \quad B \in \mathbb{R}^{2n \times 2n}, \end{aligned}$$

we have

$$\text{sign}(B) = \begin{bmatrix} I_n & X \\ 0 & I_n \end{bmatrix} \begin{bmatrix} -I_n & \\ & I_n \end{bmatrix} \begin{bmatrix} I_n & -X \\ 0 & I_n \end{bmatrix} = \begin{bmatrix} -I_n & 2X \\ 0 & I_n \end{bmatrix}.$$



The Newton iter for $\text{sign}(B)$ (**Roberts, '80**)

$$B_k = \frac{1}{2}(B_{k-1} + B_{k-1}^{-1}), \quad B_0 = B \in \mathbb{R}^{2n \times 2n}.$$

- Idea: equate **diag** and **off-diag** blocks, of

$$\text{sign}(B) = \begin{bmatrix} -I_n & 2X \\ 0 & I_n \end{bmatrix}, \quad B = \begin{bmatrix} A & W \\ 0 & -A^T \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} A^{-1} & A^{-1}W A^{-T} \\ 0 & -A^{-T} \end{bmatrix}.$$

$\rightsquigarrow$ **decoupled** Newton iter (scal. para. $\mu_k > 0$) (**Larin & Aliev, '93**)

$$A_k = \frac{1}{2}(\mu_{k-1}A_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}), \quad A_0 = A \in \mathbb{R}^{n \times n},$$
$$W_k = \frac{1}{2}(\mu_{k-1}W_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}W_{k-1}A_{k-1}^{-T}), \quad W_0 = W \in \mathbb{R}^{n \times n},$$

with $A_k \rightarrow -I_n$ and $W_k \rightarrow 2X$.



The Newton iter for $\text{sign}(B)$ (**Roberts, '80**)

$$B_k = \frac{1}{2}(B_{k-1} + B_{k-1}^{-1}), \quad B_0 = B \in \mathbb{R}^{2n \times 2n}.$$

- Idea: equate **diag** and **off-diag** blocks, of

$$\text{sign}(B) = \begin{bmatrix} -I_n & 2X \\ 0 & I_n \end{bmatrix}, \quad B = \begin{bmatrix} A & W \\ 0 & -A^T \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} A^{-1} & A^{-1}W A^{-T} \\ 0 & -A^{-T} \end{bmatrix}.$$

$\rightsquigarrow$ **decoupled** Newton iter (scal. para. $\mu_k > 0$) (**Larin & Aliev, '93**)

$$A_k = \frac{1}{2}(\mu_{k-1}A_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}), \quad A_0 = A \in \mathbb{R}^{n \times n},$$
$$W_k = \frac{1}{2}(\mu_{k-1}W_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}W_{k-1}A_{k-1}^{-T}), \quad W_0 = W \in \mathbb{R}^{n \times n},$$

with $A_k \rightarrow -I_n$ and $W_k \rightarrow 2X$.



The Newton iter for $\text{sign}(B)$ (**Roberts, '80**)

$$B_k = \frac{1}{2}(B_{k-1} + B_{k-1}^{-1}), \quad B_0 = B \in \mathbb{R}^{2n \times 2n}.$$

- Idea: equate **diag** and **off-diag** blocks, of

$$\text{sign}(B) = \begin{bmatrix} -I_n & 2X \\ 0 & I_n \end{bmatrix}, \quad B = \begin{bmatrix} A & W \\ 0 & -A^T \end{bmatrix}, \quad B^{-1} = \begin{bmatrix} A^{-1} & A^{-1}W A^{-T} \\ 0 & -A^{-T} \end{bmatrix}.$$

$\rightsquigarrow$ **decoupled** Newton iter (scal. para. $\mu_k > 0$) (**Larin & Aliev, '93**)

$$\begin{aligned} A_k &= \frac{1}{2}(\mu_{k-1}A_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}), & A_0 &= A \in \mathbb{R}^{n \times n}, \\ W_k &= \frac{1}{2}(\mu_{k-1}W_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1}W_{k-1}A_{k-1}^{-T}), & W_0 &= W \in \mathbb{R}^{n \times n}, \end{aligned}$$

with $A_k \rightarrow -I_n$ and $W_k \rightarrow 2X$.



Sign function Newton iteration

Iterating on solution factors

- Idea: **factorizing** out the subiter for W_k (**Benner & Quintana-Ortí, '99**):

$$\begin{aligned} W_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix} \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix} \begin{bmatrix} Z_{k-1}^T \\ Z_{k-1}^T A_{k-1}^{-T} \end{bmatrix} \\ &= Z_k Y_k Z_k^T, \quad W_0 = W \equiv L S L^T. \end{aligned}$$

$\rightsquigarrow$ Newton iter for Z and Y (of $X = ZY Z^T$):

$$\begin{aligned} A_k &= \frac{1}{2} (\mu_{k-1} A_{k-1} + \mu_{k-1}^{-1} A_{k-1}^{-1}), & A_0 &= A, \\ Y_k &= \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix}, & Y_0 &= S, \\ Z_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix}, & Z_0 &= L, \end{aligned}$$

with $Z_k \rightarrow Z$ and $Y_k/2 \rightarrow Y$.



Sign function Newton iteration

Iterating on solution factors

- Idea: **factorizing** out the subiter for W_k (**Benner & Quintana-Ortí, '99**):

$$\begin{aligned} W_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix} \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix} \begin{bmatrix} Z_{k-1}^T \\ Z_{k-1}^T A_{k-1}^{-T} \end{bmatrix} \\ &= Z_k Y_k Z_k^T, \quad W_0 = W \equiv L S L^T. \end{aligned}$$

$\rightsquigarrow$ Newton iter for Z and Y (of $X = ZY Z^T$):

$$\begin{aligned} A_k &= \frac{1}{2} (\mu_{k-1} A_{k-1} + \mu_{k-1}^{-1} A_{k-1}^{-1}), & A_0 &= A, \\ Y_k &= \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix}, & Y_0 &= S, \\ Z_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix}, & Z_0 &= L, \end{aligned}$$

with $Z_k \rightarrow Z$ and $Y_k/2 \rightarrow Y$.



- In each iter, two full n -by- n matrix prod saved but col of Y_k & Z_k doubled: (exp storage space):

$$\begin{aligned} A_k &= \frac{1}{2} (\mu_{k-1} A_{k-1} + \mu_{k-1}^{-1} A_{k-1}^{-1}), & A_0 &= A, \\ Y_k &= \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \\ & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix}, & Y_0 &= S, \\ Z_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix}, & Z_0 &= L, \end{aligned}$$

$\rightsquigarrow$ Rank trunc. on Z_k and Y_k (Benner & Quintana-Ortí, '99):

Subfunction $[Z, Y] = \text{RANKTRUNCLDLT}(Z, Y)$

- 1 Comput. thin QR decom $Z = QR$.
 - 2 Comput. spec decom $RYR^T = \tilde{V}\tilde{\Lambda}\tilde{V}^T$.
 - 3 $\Lambda = \text{diag}(\tilde{\lambda}_i)$, $i \in I_\lambda := \{i \mid \tilde{\lambda}_i > u\|\tilde{\Lambda}\|_1\}$ and $V = \tilde{V}(:, I_\lambda)$
 - 4 $Z = QV$, $Y = \Lambda$
-



- In each iter, two full n -by- n matrix prod saved but col of Y_k & Z_k doubled: (exp storage space):

$$\begin{aligned}
 A_k &= \frac{1}{2} (\mu_{k-1} A_{k-1} + \mu_{k-1}^{-1} A_{k-1}^{-1}), & A_0 &= A, \\
 Y_k &= \begin{bmatrix} \frac{\mu_{k-1}}{2} Y_{k-1} & \\ & \frac{1}{2\mu_{k-1}} Y_{k-1} \end{bmatrix}, & Y_0 &= S, \\
 Z_k &= \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1} Z_{k-1} \end{bmatrix}, & Z_0 &= L,
 \end{aligned}$$

$\rightsquigarrow$ Rank trunc. on Z_k and Y_k (Benner & Quintana-Ortí, '99):

Subfunction $[Z, Y] = \text{RANKTRUNCLDLT}(Z, Y)$

- 1 Comput. thin QR decom $Z = QR$.
 - 2 Comput. spec decom $RYR^T = \tilde{V}\tilde{\Lambda}\tilde{V}^T$.
 - 3 $\Lambda = \text{diag}(\tilde{\lambda}_i)$, $i \in I_\lambda := \{i \mid \tilde{\lambda}_i > u\|\tilde{\Lambda}\|_1\}$ and $V = \tilde{V}(:, I_\lambda)$
 - 4 $Z = QV$, $Y = \Lambda$
-



Algorithm: Sign function Newton iteration for LDL^T -type sol'n factor of the Lyapunov equation $AX + XA^T + W = 0$, $W = LSL^T$.

```
1  $Z_0 = L$ ,  $Y_0 = S$ ,  $A_0 = A$ 
2 for  $k \leftarrow 1$  to  $k_{\max}$  do
3    $A_k = \frac{1}{2}(\mu_{k-1}A_{k-1} + \mu_{k-1}^{-1}A_{k-1}^{-1})$  [*] with  $\mu_{k-1}$  a form of scaling.
4    $Z_k = \begin{bmatrix} Z_{k-1} & A_{k-1}^{-1}Z_{k-1} \end{bmatrix}$ 
5    $Y_k = \frac{1}{2} \text{blkdiag}(\mu_{k-1}Y_{k-1}, \mu_{k-1}^{-1}Y_{k-1})$ 
6   if  $\text{size}(Z_k, 2) > \rho n$  then  $[Z_k, Y_k] = \text{RANKTRUNC LDLT}(Z_k, Y_k)$ 
7   if  $\|A_k + I_n\|_1 \leq \tau$  then
8      $\quad$   $\quad$  Terminate after two more iterations.
9  $Y = Y_k/2$ 
```

- Parameters: cge. tol: $\tau = 10\sqrt{nu}$, rank trunc. tol: $\rho \ll 1$
- [*]Dominant cost each iteration: $2n^3$



In each **refinement (outer) iter**:

- Three main steps
 - $O(n^2)$ flops for res decomp. and sol'n factor update
 - Assume a flop at (higher) u_r or u_c is no more expensive than n flops at u_s
 - Dominant cost (solver step): $2kn^3$ for k **Newton (inner) iters**
- Cost: $2(k_0 + k_1 + \dots + k_i)n^3$ flops in u_s . k_ℓ the number of **Newton (inner) iters** at ℓ th **refinement iter**.
- A **cost balance**: lower precision $u_s \rightsquigarrow$ slower cge. of **Newton (inner) iter**, requiring more inner iters k_i .



In each **refinement (outer) iter**:

- Three main steps
 - $O(n^2)$ flops for res decomp. and sol'n factor update
 - Assume a flop at (higher) u_r or u_c is no more expensive than n flops at u_s
 - Dominant cost (solver step): $2kn^3$ for k **Newton (inner) iters**
- Cost: $2(k_0 + k_1 + \dots + k_i)n^3$ flops in u_s . k_ℓ the number of **Newton (inner) iters** at ℓ th **refinement iter**.
 - A **cost balance**: lower precision $u_s \rightsquigarrow$ slower cge. of **Newton (inner) iter**, requiring more inner iters k_i .



In each **refinement (outer) iter**:

- Three main steps
 - $O(n^2)$ flops for res decomp. and sol'n factor update
 - Assume a flop at (higher) u_r or u_c is no more expensive than n flops at u_s
 - Dominant cost (solver step): $2kn^3$ for k **Newton (inner) iters**
- Cost: $2(k_0 + k_1 + \dots + k_i)n^3$ flops in u_s . k_ℓ the number of **Newton (inner) iters** at ℓ th **refinement iter**.
 - A **cost balance**: lower precision $u_s \rightsquigarrow$ slower cge. of **Newton (inner) iter**, requiring more inner iters k_i .



- Fact: in each **refinement (outer) iter**, same sequence

$$A_1^{-1}, A_2^{-1}, \dots, \quad A_0 = A.$$

↪ Key idea: reuse $\hat{A}_1^{-1}, \hat{A}_2^{-1}, \dots$ across different refinement iters

- Cost: $\frac{2}{3} \sum_{\ell=0}^i k_{\ell} n^3$ $2 \max_{0 \leq \ell \leq i} k_{\ell} n^3$ flops in precision u_s . k_{ℓ} the number of **Newton (inner) iterations** at ℓ th **refinement iteration**.

- Reduced comput. cost & increased data movement

↪ applicable to memory-efficient systems

- Quantity of interest** in comput. cost comparison:

$$k^{\sigma} =: \sum_{\ell=0}^i k_{\ell} \quad \text{or} \quad k^{\max} =: \max_{0 \leq \ell \leq i} k_{\ell}.$$

Compare flops in diff solver precisions:

- half/double: $k_h^{\sigma}/k_d^{\sigma}$ or $k_h^{\max}/k_d^{\max}$
- single/double: $k_s^{\sigma}/k_d^{\sigma}$ or $k_s^{\max}/k_d^{\max}$



- Fact: in each **refinement (outer) iter**, same sequence

$$A_1^{-1}, A_2^{-1}, \dots, \quad A_0 = A.$$

↪ Key idea: reuse $\hat{A}_1^{-1}, \hat{A}_2^{-1}, \dots$ across different refinement iters

- Cost: ~~$2 \sum_{\ell=0}^i k_{\ell} n^3$~~ $2 \max_{0 \leq \ell \leq i} k_{\ell} n^3$ flops in precision u_s . k_{ℓ} the number of **Newton (inner) iterations** at ℓ th **refinement iteration**.

- Reduced comput. cost & increased data movement

↪ applicable to memory-efficient systems

- Quantity of interest** in comput. cost comparison:

$$k^{\sigma} =: \sum_{\ell=0}^i k_{\ell} \quad \text{or} \quad k^{\max} =: \max_{0 \leq \ell \leq i} k_{\ell}.$$

Compare flops in diff solver precisions:

- half/double: $k_h^{\sigma}/k_d^{\sigma}$ or $k_h^{\max}/k_d^{\max}$
- single/double: $k_s^{\sigma}/k_d^{\sigma}$ or $k_s^{\max}/k_d^{\max}$



- Fact: in each **refinement (outer) iter**, same sequence

$$A_1^{-1}, A_2^{-1}, \dots, \quad A_0 = A.$$

↪ Key idea: reuse $\hat{A}_1^{-1}, \hat{A}_2^{-1}, \dots$ across different refinement iters

- Cost: ~~$\frac{2}{3} \sum_{\ell=0}^i k_{\ell} n^3$~~ $2 \max_{0 \leq \ell \leq i} k_{\ell} n^3$ flops in precision u_s . k_{ℓ} the number of **Newton (inner) iterations** at ℓ th **refinement iteration**.

- Reduced comput. cost & increased data movement

↪ applicable to memory-efficient systems

- Quantity of interest** in comput. cost comparison:

$$k^{\sigma} =: \sum_{\ell=0}^i k_{\ell} \quad \text{or} \quad k^{\max} =: \max_{0 \leq \ell \leq i} k_{\ell}.$$

Compare flops in diff solver precisions:

- half/double: $k_h^{\sigma}/k_d^{\sigma}$ or $k_h^{\max}/k_d^{\max}$
- single/double: $k_s^{\sigma}/k_d^{\sigma}$ or $k_s^{\max}/k_d^{\max}$



- Fact: in each **refinement (outer) iter**, same sequence

$$A_1^{-1}, A_2^{-1}, \dots, \quad A_0 = A.$$

↪ Key idea: reuse $\hat{A}_1^{-1}, \hat{A}_2^{-1}, \dots$ across different refinement iters

- Cost: $\frac{2 \sum_{\ell=0}^i k_{\ell} n^3}{2 \max_{0 \leq \ell \leq i} k_{\ell} n^3}$ flops in precision u_s . k_{ℓ} the number of **Newton (inner) iterations** at ℓ th **refinement iteration**.

- Reduced comput. cost & increased data movement

↪ applicable to memory-efficient systems

- Quantity of interest** in comput. cost comparison:

$$k^{\sigma} =: \sum_{\ell=0}^i k_{\ell} \quad \text{or} \quad k^{\max} =: \max_{0 \leq \ell \leq i} k_{\ell}.$$

Compare flops in diff solver precisions:

- half/double: $k_h^{\sigma}/k_d^{\sigma}$ or $k_h^{\max}/k_d^{\max}$
- single/double: $k_s^{\sigma}/k_d^{\sigma}$ or $k_s^{\max}/k_d^{\max}$



1. Background

- Low-rank Lyapunov equations
- Floating-point arithmetics
- Use of low and mixed precisions

2. Mixed-precision iterative refinement for linear systems

- For $Ax = b$
- For low-rank Lyapunov equation
- Rounding error analysis

3. Sign function Newton iteration as solver

- Sign function Newton iteration
- Cost analysis

4. Numerical experiments



- bf16, fp32, fp64 used in MATLAB, bf16 simulated by chop (**Higham & Pranesh, '19**)
- We measure

$$\text{Res}(\hat{Z}, \hat{Y}) = \frac{\|A\hat{Z}\hat{Y}\hat{Z}^T + \hat{Z}\hat{Y}\hat{Z}^T A^T + W\|_F}{\|W\|_F + 2\|\hat{Z}\hat{Y}\hat{Z}^T\|_F\|A\|_F},$$

computed in fp64.

- Parameters:
 - mplr cge. tol = nu
 - Max (outer) refinement steps = 50
 - Max (inner) Newton iters = 50



Numerical experiments

Tests on pseudorandom matrices

		$u = \text{fp32}$						$u = \text{fp64}$								
n	cond	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.6e-6	4(2)	6	6.4e-7	2(2)	6	1.0e-15	12(2)	15	6.8e-15	6(2)	18	8.4e-17	4(4)	15
	3.2e0	3.9e-6	4(2)	10	3.3e-7	3(3)	9	1.3e-16	14(2)	24	4.7e-15	9(3)	25	9.1e-17	5(5)	24
	1.0e1	2.8e-6	6(2)	12	2.6e-8	4(4)	12	2.5e-15	20(2)	33	1.2e-16	12(4)	33	1.8e-15	5(5)	33
	3.2e1	2.9e-6	10(2)	13	3.0e-7	4(4)	13	7.8e-15	36(2)	42	4.4e-15	12(4)	44	6.7e-17	6(6)	42
	1.0e2	4.1e-6	18(2)	16	3.3e-8	5(5)	15	6.8e-15	74(2)	49	2.9e-16	15(5)	49	6.2e-17	6(6)	49
	3.2e2	3.3e-6	18(3)	16	2.6e-8	5(5)	16	8.9e-15	66(3)	62	2.6e-16	15(5)	56	7.7e-17	7(7)	56
	1.0e3	1.1e-4	33(3)	–	4.4e-8	5(5)	16	1.3e-4	30(3)	–	3.2e-16	15(5)	62	8.8e-17	7(7)	62
	3.2e3	1.0e-3	15(5)	–	3.3e-8	5(5)	16	7.1e-4	15(5)	–	1.3e-15	15(5)	68	9.7e-17	7(7)	68
1000	1.3e0	2.5e-7	4(2)	6	2.4e-7	2(2)	6	5.2e-14	10(2)	22	1.4e-15	6(2)	12	1.5e-17	4(4)	12
	3.2e0	1.3e-6	4(2)	6	1.3e-7	3(3)	6	2.7e-14	12(2)	33	1.8e-15	9(3)	22	1.4e-17	5(5)	22
	1.0e1	1.2e-5	4(2)	9	7.3e-6	3(3)	9	8.0e-15	18(2)	31	9.6e-15	12(3)	31	8.6e-16	5(5)	31
	3.2e1	2.5e-5	6(2)	11	1.2e-7	4(4)	10	7.8e-14	32(2)	90	1.2e-15	12(4)	39	2.1e-17	6(6)	39
	1.0e2	3.6e-5	8(2)	13	5.7e-7	4(4)	11	6.6e-14	72(2)	48	1.3e-16	16(4)	48	2.7e-17	6(6)	48
	3.2e2	2.0e-4	102(2)	–	5.8e-9	5(5)	12	2.0e-4	102(2)	–	8.5e-17	15(5)	55	2.6e-17	7(7)	55
	1.0e3	1.1e-4	21(3)	–	6.9e-9	5(5)	12	1.1e-4	21(3)	–	1.2e-16	15(5)	61	2.6e-17	7(7)	61
	3.2e3	5.3e-5	12(3)	16	1.0e-8	5(5)	10	1.5e-7	84(3)	–	1.7e-15	15(5)	77	2.6e-17	7(7)	69

1. iter and the rank generally increase as cond grow
2. Potential instability of solver
3. $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma \approx 2$ if well conditioned; $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$
4. Potential speed-up: fp16/bf16 2–8× faster than fp32 and 4–16× faster than fp64 (e.g., NVIDIA's Tensor Cores (Haidar, Tomov, & Dongarra, '19))



Numerical experiments

Tests on pseudorandom matrices

		$u = \text{fp32}$						$u = \text{fp64}$								
n	cond	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.6e-6	4(2)	6	6.4e-7	2(2)	6	1.0e-15	12(2)	15	6.8e-15	6(2)	18	8.4e-17	4(4)	15
	3.2e0	3.9e-6	4(2)	10	3.3e-7	3(3)	9	1.3e-16	14(2)	24	4.7e-15	9(3)	25	9.1e-17	5(5)	24
	1.0e1	2.8e-6	6(2)	12	2.6e-8	4(4)	12	2.5e-15	20(2)	33	1.2e-16	12(4)	33	1.8e-15	5(5)	33
	3.2e1	2.9e-6	10(2)	13	3.0e-7	4(4)	13	7.8e-15	36(2)	42	4.4e-15	12(4)	44	6.7e-17	6(6)	42
	1.0e2	4.1e-6	18(2)	16	3.3e-8	5(5)	15	6.8e-15	74(2)	49	2.9e-16	15(5)	49	6.2e-17	6(6)	49
	3.2e2	3.3e-6	18(3)	16	2.6e-8	5(5)	16	8.9e-15	66(3)	62	2.6e-16	15(5)	56	7.7e-17	7(7)	56
	1.0e3	1.1e-4	33(3)	–	4.4e-8	5(5)	16	1.3e-4	30(3)	–	3.2e-16	15(5)	62	8.8e-17	7(7)	62
	3.2e3	1.0e-3	15(5)	–	3.3e-8	5(5)	16	7.1e-4	15(5)	–	1.3e-15	15(5)	68	9.7e-17	7(7)	68
1000	1.3e0	2.5e-7	4(2)	6	2.4e-7	2(2)	6	5.2e-14	10(2)	22	1.4e-15	6(2)	12	1.5e-17	4(4)	12
	3.2e0	1.3e-6	4(2)	6	1.3e-7	3(3)	6	2.7e-14	12(2)	33	1.8e-15	9(3)	22	1.4e-17	5(5)	22
	1.0e1	1.2e-5	4(2)	9	7.3e-6	3(3)	9	8.0e-15	18(2)	31	9.6e-15	12(3)	31	8.6e-16	5(5)	31
	3.2e1	2.5e-5	6(2)	11	1.2e-7	4(4)	10	7.8e-14	32(2)	90	1.2e-15	12(4)	39	2.1e-17	6(6)	39
	1.0e2	3.6e-5	8(2)	13	5.7e-7	4(4)	11	6.6e-14	72(2)	48	1.3e-16	16(4)	48	2.7e-17	6(6)	48
	3.2e2	2.0e-4	102(2)	–	5.8e-9	5(5)	12	2.0e-4	102(2)	–	8.5e-17	15(5)	55	2.6e-17	7(7)	55
	1.0e3	1.1e-4	21(3)	–	6.9e-9	5(5)	12	1.1e-4	21(3)	–	1.2e-16	15(5)	61	2.6e-17	7(7)	61
	3.2e3	5.3e-5	12(3)	16	1.0e-8	5(5)	10	1.5e-7	84(3)	–	1.7e-15	15(5)	77	2.6e-17	7(7)	69

1. iter and the rank generally increase as cond grow
2. Potential instability of solver
3. $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma \approx 2$ if well conditioned; $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$
4. Potential speed-up: fp16/bf16 2–8× faster than fp32 and 4–16× faster than fp64 (e.g., NVIDIA's Tensor Cores (Haidar, Tomov, & Dongarra, '19))



Numerical experiments

Tests on pseudorandom matrices

		$u = \text{fp32}$						$u = \text{fp64}$								
n	cond	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.6e-6	4(2)	6	6.4e-7	2(2)	6	1.0e-15	12(2)	15	6.8e-15	6(2)	18	8.4e-17	4(4)	15
	3.2e0	3.9e-6	4(2)	10	3.3e-7	3(3)	9	1.3e-16	14(2)	24	4.7e-15	9(3)	25	9.1e-17	5(5)	24
	1.0e1	2.8e-6	6(2)	12	2.6e-8	4(4)	12	2.5e-15	20(2)	33	1.2e-16	12(4)	33	1.8e-15	5(5)	33
	3.2e1	2.9e-6	10(2)	13	3.0e-7	4(4)	13	7.8e-15	36(2)	42	4.4e-15	12(4)	44	6.7e-17	6(6)	42
	1.0e2	4.1e-6	18(2)	16	3.3e-8	5(5)	15	6.8e-15	74(2)	49	2.9e-16	15(5)	49	6.2e-17	6(6)	49
	3.2e2	3.3e-6	18(3)	16	2.6e-8	5(5)	16	8.9e-15	66(3)	62	2.6e-16	15(5)	56	7.7e-17	7(7)	56
	1.0e3	1.1e-4	33(3)	16	4.4e-8	5(5)	16	1.3e-4	30(3)	62	3.2e-16	15(5)	62	8.8e-17	7(7)	62
	3.2e3	1.0e-3	15(5)	16	3.3e-8	5(5)	16	7.1e-4	15(5)	68	1.3e-15	15(5)	68	9.7e-17	7(7)	68
1000	1.3e0	2.5e-7	4(2)	6	2.4e-7	2(2)	6	5.2e-14	10(2)	22	1.4e-15	6(2)	12	1.5e-17	4(4)	12
	3.2e0	1.3e-6	4(2)	6	1.3e-7	3(3)	6	2.7e-14	12(2)	33	1.8e-15	9(3)	22	1.4e-17	5(5)	22
	1.0e1	1.2e-5	4(2)	9	7.3e-6	3(3)	9	8.0e-15	18(2)	31	9.6e-15	12(3)	31	8.6e-16	5(5)	31
	3.2e1	2.5e-5	6(2)	11	1.2e-7	4(4)	10	7.8e-14	32(2)	90	1.2e-15	12(4)	39	2.1e-17	6(6)	39
	1.0e2	3.6e-5	8(2)	13	5.7e-7	4(4)	11	6.6e-14	72(2)	48	1.3e-16	16(4)	48	2.7e-17	6(6)	48
	3.2e2	2.0e-4	102(2)	16	5.8e-9	5(5)	12	2.0e-4	102(2)	62	8.5e-17	15(5)	55	2.6e-17	7(7)	55
	1.0e3	1.1e-4	21(3)	16	6.9e-9	5(5)	12	1.1e-4	21(3)	62	1.2e-16	15(5)	61	2.6e-17	7(7)	61
	3.2e3	5.3e-5	12(3)	16	1.0e-8	5(5)	10	1.5e-7	84(3)	68	1.7e-15	15(5)	77	2.6e-17	7(7)	69

1. iter and the rank generally increase as cond grow
2. Potential instability of solver
3. $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma \approx 2$ if well conditioned; $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$
4. Potential speed-up: fp16/bf16 2–8× faster than fp32 and 4–16× faster than fp64 (e.g., NVIDIA's Tensor Cores (Haidar, Tomov, & Dongarra, '19))



Numerical experiments

Tests on pseudorandom matrices

		$u = \text{fp32}$						$u = \text{fp64}$								
n	cond	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.6e-6	4(2)	6	6.4e-7	2(2)	6	1.0e-15	12(2)	15	6.8e-15	6(2)	18	8.4e-17	4(4)	15
	3.2e0	3.9e-6	4(2)	10	3.3e-7	3(3)	9	1.3e-16	14(2)	24	4.7e-15	9(3)	25	9.1e-17	5(5)	24
	1.0e1	2.8e-6	6(2)	12	2.6e-8	4(4)	12	2.5e-15	20(2)	33	1.2e-16	12(4)	33	1.8e-15	5(5)	33
	3.2e1	2.9e-6	10(2)	13	3.0e-7	4(4)	13	7.8e-15	36(2)	42	4.4e-15	12(4)	44	6.7e-17	6(6)	42
	1.0e2	4.1e-6	18(2)	16	3.3e-8	5(5)	15	6.8e-15	74(2)	49	2.9e-16	15(5)	49	6.2e-17	6(6)	49
	3.2e2	3.3e-6	18(3)	16	2.6e-8	5(5)	16	8.9e-15	66(3)	62	2.6e-16	15(5)	56	7.7e-17	7(7)	56
	1.0e3	1.1e-4	33(3)	–	4.4e-8	5(5)	16	1.3e-4	30(3)	–	3.2e-16	15(5)	62	8.8e-17	7(7)	62
	3.2e3	1.0e-3	15(5)	–	3.3e-8	5(5)	16	7.1e-4	15(5)	–	1.3e-15	15(5)	68	9.7e-17	7(7)	68
1000	1.3e0	2.5e-7	4(2)	6	2.4e-7	2(2)	6	5.2e-14	10(2)	22	1.4e-15	6(2)	12	1.5e-17	4(4)	12
	3.2e0	1.3e-6	4(2)	6	1.3e-7	3(3)	6	2.7e-14	12(2)	33	1.8e-15	9(3)	22	1.4e-17	5(5)	22
	1.0e1	1.2e-5	4(2)	9	7.3e-6	3(3)	9	8.0e-15	18(2)	31	9.6e-15	12(3)	31	8.6e-16	5(5)	31
	3.2e1	2.5e-5	6(2)	11	1.2e-7	4(4)	10	7.8e-14	32(2)	90	1.2e-15	12(4)	39	2.1e-17	6(6)	39
	1.0e2	3.6e-5	8(2)	13	5.7e-7	4(4)	11	6.6e-14	72(2)	48	1.3e-16	16(4)	48	2.7e-17	6(6)	48
	3.2e2	2.0e-4	102(2)	–	5.8e-9	5(5)	12	2.0e-4	102(2)	–	8.5e-17	15(5)	55	2.6e-17	7(7)	55
	1.0e3	1.1e-4	21(3)	–	6.9e-9	5(5)	12	1.1e-4	21(3)	–	1.2e-16	15(5)	61	2.6e-17	7(7)	61
	3.2e3	5.3e-5	12(3)	16	1.0e-8	5(5)	10	1.5e-7	84(3)	–	1.7e-15	15(5)	77	2.6e-17	7(7)	69

1. iter and the rank generally increase as cond grow
2. Potential instability of solver
3. $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma \approx 2$ if **well conditioned**; $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$
4. Potential speed-up: fp16/bf16 2–8× faster than fp32 and 4–16× faster than fp64 (e.g., NVIDIA's Tensor Cores (Haidar, Tomov, & Dongarra, '19))



Numerical experiments

Tests on pseudorandom matrices

		u = fp32						u = fp64								
n	cond	u _s = bf16			u _s = fp32			u _s = bf16			u _s = fp32			u _s = fp64		
		Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
100	1.3e0	1.6e-6	4(2)	6	6.4e-7	2(2)	6	1.0e-15	12(2)	15	6.8e-15	6(2)	18	8.4e-17	4(4)	15
	3.2e0	3.9e-6	4(2)	10	3.3e-7	3(3)	9	1.3e-16	14(2)	24	4.7e-15	9(3)	25	9.1e-17	5(5)	24
	1.0e1	2.8e-6	6(2)	12	2.6e-8	4(4)	12	2.5e-15	20(2)	33	1.2e-16	12(4)	33	1.8e-15	5(5)	33
	3.2e1	2.9e-6	10(2)	13	3.0e-7	4(4)	13	7.8e-15	36(2)	42	4.4e-15	12(4)	44	6.7e-17	6(6)	42
	1.0e2	4.1e-6	18(2)	16	3.3e-8	5(5)	15	6.8e-15	74(2)	49	2.9e-16	15(5)	49	6.2e-17	6(6)	49
	3.2e2	3.3e-6	18(3)	16	2.6e-8	5(5)	16	8.9e-15	66(3)	62	2.6e-16	15(5)	56	7.7e-17	7(7)	56
	1.0e3	1.1e-4	33(3)	–	4.4e-8	5(5)	16	1.3e-4	30(3)	–	3.2e-16	15(5)	62	8.8e-17	7(7)	62
	3.2e3	1.0e-3	15(5)	–	3.3e-8	5(5)	16	7.1e-4	15(5)	–	1.3e-15	15(5)	68	9.7e-17	7(7)	68
1000	1.3e0	2.5e-7	4(2)	6	2.4e-7	2(2)	6	5.2e-14	10(2)	22	1.4e-15	6(2)	12	1.5e-17	4(4)	12
	3.2e0	1.3e-6	4(2)	6	1.3e-7	3(3)	6	2.7e-14	12(2)	33	1.8e-15	9(3)	22	1.4e-17	5(5)	22
	1.0e1	1.2e-5	4(2)	9	7.3e-6	3(3)	9	8.0e-15	18(2)	31	9.6e-15	12(3)	31	8.6e-16	5(5)	31
	3.2e1	2.5e-5	6(2)	11	1.2e-7	4(4)	10	7.8e-14	32(2)	90	1.2e-15	12(4)	39	2.1e-17	6(6)	39
	1.0e2	3.6e-5	8(2)	13	5.7e-7	4(4)	11	6.6e-14	72(2)	48	1.3e-16	16(4)	48	2.7e-17	6(6)	48
	3.2e2	2.0e-4	102(2)	–	5.8e-9	5(5)	12	2.0e-4	102(2)	–	8.5e-17	15(5)	55	2.6e-17	7(7)	55
	1.0e3	1.1e-4	21(3)	–	6.9e-9	5(5)	12	1.1e-4	21(3)	–	1.2e-16	15(5)	61	2.6e-17	7(7)	61
	3.2e3	5.3e-5	12(3)	16	1.0e-8	5(5)	10	1.5e-7	84(3)	–	1.7e-15	15(5)	77	2.6e-17	7(7)	69

1. iter and the rank generally increase as cond grow
2. Potential instability of solver
3. $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma \approx 2$ if **well conditioned**; $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$
4. Potential speed-up: fp16/bf16 2–8× faster than fp32 and 4–16× faster than fp64 (e.g., NVIDIA's Tensor Cores (**Haidar, Tomov, & Dongarra, '19**))



Table: Summary of test problems from SLICOT library (**Chahlaoui & Dooren, '02**).

Dataset	n	m	Nonzeros	$\kappa_F(A)$	$\kappa_{\text{sign}}(B)$
beam	348	1	60,726	1.2e7	1.4e9
build	48	1	1,176	7.5e4	2.0e6
CDplayer	120	2	240	1.4e5	4.1e10
eady	598	1	357,406	3.6e3	3.8e6
fom	1,006	1	1,012	2.3e4	5.2e5
heat-cont	200	1	598	1.5e5	1.0e4
iss	270	3	405	2.5e5	1.4e11
pde	84	1	382	1.2e2	1.6e2
random	200	1	2132	3.2e3	2.3e10

- $\kappa_{\text{sign}}(B) \rightsquigarrow$ **stability** of sign function Newton iteration in floating-point arithmetic (**Higham, '08**).



Dataset	$u = \text{fp32}$						$u = \text{fp64}$								
	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
beam	3.1e-3	45(15)	–	2.2e-7	14(14)	54	9.2e-4	80(16)	–	2.0e-14	70(14)	143	3.2e-16	16(16)	134
build	1.5e-4	36(12)	–	4.5e-8	14(14)	35	1.5e-4	60(12)	–	7.6e-16	70(14)	48	6.0e-17	15(15)	48
CDplayer	4.1e-6	8(8)	2	7.4e-8	16(16)	10	3.3e-8	128(8)	–	1.8e-17	64(16)	116	6.7e-17	18(18)	116
eady	2.5e-5	71(12)	21	9.2e-8	15(15)	12	2.2e-7	263(12)	–	7.3e-12	60(15)	–	2.3e-16	16(16)	88
fom	2.4e-3	12(3)	–	2.6e-7	13(13)	12	2.4e-3	12(3)	–	3.6e-15	39(13)	29	5.7e-16	15(15)	27
heat-cont	4.1e-4	12(4)	–	2.4e-8	7(7)	10	3.9e-4	12(4)	–	1.0e-16	28(7)	26	5.3e-17	9(9)	26
iss	1.2e-5	14(14)	6	1.8e-8	21(21)	46	3.1e-5	60(15)	–	1.8e-8	63(21)	–	2.4e-17	23(23)	223
pde [*]	3.6e-7	8(2)	5	2.8e-8	4(4)	5	3.4e-15	18(2)	11	1.4e-16	12(4)	11	5.4e-17	6(6)	11
random	6.4e-5	91(7)	–	4.8e-8	15(15)	2	9.9e-5	60(7)	–	4.6e-16	60(15)	24	9.8e-17	16(16)	24

[*] Problem of cond $\leq 10^3$.

1. Cge. only on *well conditioned* problems with $u_s = \text{bf16}$.
2. Div. with $u_s = \text{fp32}$ and $u = \text{fp64}$ on two problems $\kappa_{\text{sign}}(B) \sim 10^6, 10^{10}$
3. In most cge. cases, $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma > 2$, but $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$



Dataset	$u = \text{fp32}$						$u = \text{fp64}$								
	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
beam	3.1e-3	45(15)	–	2.2e-7	14(14)	54	9.2e-4	80(16)	–	2.0e-14	70(14)	143	3.2e-16	16(16)	134
build	1.5e-4	36(12)	–	4.5e-8	14(14)	35	1.5e-4	60(12)	–	7.6e-16	70(14)	48	6.0e-17	15(15)	48
CDplayer	4.1e-6	8(8)	2	7.4e-8	16(16)	10	3.3e-8	128(8)	–	1.8e-17	64(16)	116	6.7e-17	18(18)	116
eady	2.5e-5	71(12)	21	9.2e-8	15(15)	12	2.2e-7	263(12)	–	7.3e-12	60(15)	–	2.3e-16	16(16)	88
fom	2.4e-3	12(3)	–	2.6e-7	13(13)	12	2.4e-3	12(3)	–	3.6e-15	39(13)	29	5.7e-16	15(15)	27
heat-cont	4.1e-4	12(4)	–	2.4e-8	7(7)	10	3.9e-4	12(4)	–	1.0e-16	28(7)	26	5.3e-17	9(9)	26
iss	1.2e-5	14(14)	6	1.8e-8	21(21)	46	3.1e-5	60(15)	–	1.8e-8	63(21)	–	2.4e-17	23(23)	223
pde [*]	3.6e-7	8(2)	5	2.8e-8	4(4)	5	3.4e-15	18(2)	11	1.4e-16	12(4)	11	5.4e-17	6(6)	5
random	6.4e-5	91(7)	–	4.8e-8	15(15)	2	9.9e-5	60(7)	–	4.6e-16	60(15)	24	9.8e-17	16(16)	24

[*] Problem of cond $\leq 10^3$.

1. Cge. only on *well conditioned* problems with $u_s = \text{bf16}$.
2. Div. with $u_s = \text{fp32}$ and $u = \text{fp64}$ on two problems $\kappa_{\text{sign}}(B) \sim 10^6, 10^{10}$
3. In most cge. cases, $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma > 2$, but $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$



Dataset	$u = \text{fp32}$						$u = \text{fp64}$								
	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
beam	3.1e-3	45(15)	–	2.2e-7	14(14)	54	9.2e-4	80(16)	–	2.0e-14	70(14)	143	3.2e-16	16(16)	134
build	1.5e-4	36(12)	–	4.5e-8	14(14)	35	1.5e-4	60(12)	–	7.6e-16	70(14)	48	6.0e-17	15(15)	48
CDplayer	4.1e-6	8(8)	2	7.4e-8	16(16)	10	3.3e-8	128(8)	–	1.8e-17	64(16)	116	6.7e-17	18(18)	116
eady	2.5e-5	71(12)	21	9.2e-8	15(15)	12	2.2e-7	263(12)	–	7.3e-12	60(15)	–	2.3e-16	16(16)	88
fom	2.4e-3	12(3)	–	2.6e-7	13(13)	12	2.4e-3	12(3)	–	3.6e-15	39(13)	29	5.7e-16	15(15)	27
heat-cont	4.1e-4	12(4)	–	2.4e-8	7(7)	10	3.9e-4	12(4)	–	1.0e-16	28(7)	26	5.3e-17	9(9)	26
iss	1.2e-5	14(14)	6	1.8e-8	21(21)	46	3.1e-5	60(15)	–	1.8e-8	63(21)	–	2.4e-17	23(23)	223
pde [*]	3.6e-7	8(2)	5	2.8e-8	4(4)	5	3.4e-15	18(2)	11	1.4e-16	12(4)	11	5.4e-17	6(6)	11
random	6.4e-5	91(7)	–	4.8e-8	15(15)	2	9.9e-5	60(7)	–	4.6e-16	60(15)	24	9.8e-17	16(16)	24

[*] Problem of cond $\leq 10^3$.

1. Cge. only on *well conditioned* problems with $u_s = \text{bf16}$.
2. Div. with $u_s = \text{fp32}$ and $u = \text{fp64}$ on two problems $\kappa_{\text{sign}}(B) \sim 10^6, 10^{10}$
3. In most cge. cases, $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma > 2$, but $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$



Dataset	$u = \text{fp32}$						$u = \text{fp64}$								
	$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{bf16}$			$u_s = \text{fp32}$			$u_s = \text{fp64}$		
	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank	Res	iter	rank
beam	3.1e-3	45(15)	–	2.2e-7	14(14)	54	9.2e-4	80(16)	–	2.0e-14	70(14)	143	3.2e-16	16(16)	134
build	1.5e-4	36(12)	–	4.5e-8	14(14)	35	1.5e-4	60(12)	–	7.6e-16	70(14)	48	6.0e-17	15(15)	48
CDplayer	4.1e-6	8(8)	2	7.4e-8	16(16)	10	3.3e-8	128(8)	–	1.8e-17	64(16)	116	6.7e-17	18(18)	116
eady	2.5e-5	71(12)	21	9.2e-8	15(15)	12	2.2e-7	263(12)	–	7.3e-12	60(15)	–	2.3e-16	16(16)	88
fom	2.4e-3	12(3)	–	2.6e-7	13(13)	12	2.4e-3	12(3)	–	3.6e-15	39(13)	29	5.7e-16	15(15)	27
heat-cont	4.1e-4	12(4)	–	2.4e-8	7(7)	10	3.9e-4	12(4)	–	1.0e-16	28(7)	26	5.3e-17	9(9)	26
iss	1.2e-5	14(14)	6	1.8e-8	21(21)	46	3.1e-5	60(15)	–	1.8e-8	63(21)	–	2.4e-17	23(23)	223
pde [*]	3.6e-7	8(2)	5	2.8e-8	4(4)	5	3.4e-15	18(2)	11	1.4e-16	12(4)	11	5.4e-17	6(6)	11
random	6.4e-5	91(7)	–	4.8e-8	15(15)	2	9.9e-5	60(7)	–	4.6e-16	60(15)	24	9.8e-17	16(16)	24

[*] Problem of cond $\leq 10^3$.

1. Cge. only on *well conditioned* problems with $u_s = \text{bf16}$.
2. Div. with $u_s = \text{fp32}$ and $u = \text{fp64}$ on two problems $\kappa_{\text{sign}}(B) \sim 10^6, 10^{10}$
3. In most cge. cases, $k_h^\sigma/k_s^\sigma, k_s^\sigma/k_d^\sigma > 2$, but $k_h^{\max}/k_s^{\max}, k_s^{\max}/k_d^{\max} \leq 1$



- Formalized **mixed-precision iterative refinement** framework for low-rank Lyapunov equations (Chol/LDL^T-type).
- **Rounding error analysis** for precisions & algorithmic parameters
- Reduced precisions, such as **half precision**, may be used to accelerate the solution of *well conditioned* Lyapunov equations, $\text{cond} \lesssim 1/u_s$.

► P. Benner and X. Liu. Mixed-precision iterative refinement for low-rank Lyapunov equations. preprint, [https:// ongoing work, coming soon](https://ongoingwork.comingsoon).

Next?

- Hardware implementation & performance optimization
- Reduced precision with ADI/Krylov-based solver inside mpIR?
- Inexact solvers, such as Kleinman–Newton iteration?

Thank you for your attention!



- Formalized **mixed-precision iterative refinement** framework for low-rank Lyapunov equations (Chol/LDL^T-type).
- **Rounding error analysis** for precisions & algorithmic parameters
- Reduced precisions, such as **half precision**, may be used to accelerate the solution of *well conditioned* Lyapunov equations, $\text{cond} \lesssim 1/u_s$.

► P. Benner and X. Liu. Mixed-precision iterative refinement for low-rank Lyapunov equations. preprint, [https:// ongoing work, coming soon](https://ongoingwork.comingsoon).

Next?

- Hardware implementation & performance optimization
- Reduced precision with ADI/Krylov-based solver inside mpIR?
- Inexact solvers, such as Kleinman–Newton iteration?

Thank you for your attention!



Peter Benner, Pablo Ezzatti, Daniel Kressner, Enrique S. Quintana-Ortí, and Alfredo Remón. (Benner et al.)

A mixed-precision algorithm for the solution of Lyapunov equations on hybrid CPU–GPU platforms.

Parallel Comput., 37:439–450, 2011.



Peter Benner and Enrique S. Quintana-Ortí.

Solving stable generalized Lyapunov equations with the matrix sign function.

Numer. Algorithms, 20:75–100, 1999.



Erin Carson and Nicholas J. Higham.

Accelerating the solution of linear systems by iterative refinement in three precisions.

SIAM J. Sci. Comput., 40(2):A817–A847, 2018.



Younès Chahlaoui and Paul Van Dooren.

A collection of benchmark examples for model reduction of linear time invariant dynamical systems.

SLICOT Working Note 2002-2: February 2002.



James W. Demmel.

Applied Numerical Linear Algebra.

SIAM, Philadelphia, PA, USA, 1997. xi+416 pp. ISBN 978-0-89871-389-3.



Gene H. Golub and Charles F. Van Loan.

Matrix Computations.

Johns Hopkins University Press, Baltimore, MD, USA, 4th ed., 2013. xxi+756 pp. ISBN 978-1-4214-0794-4.



Ming Gu and Stanley C. Eisenstat.

A divide-and-conquer algorithm for the symmetric tridiagonal eigenproblem.

SIAM J. Matrix. Anal. Appl., 16(1):172–191, 1995.



Azzam Haidar, Stanimire Tomov, and Jack Dongarra.

Towards half-precision computation for complex matrices: A case study for mixed precision solvers on GPUs.

In *2019 IEEE/ACM 10th Workshop on Latest Advances in Scalable Algorithms for Large-Scale Systems (ScalA)*, Denver, CO, USA, 2019, pages 17–24. IEEE.



Nicholas J. Higham.

Accuracy and Stability of Numerical Algorithms.

SIAM, Philadelphia, PA, USA, 2nd ed., 2002. xxvii+663 pp. ISBN 978-0-89871-521-7.



Nicholas J. Higham.

Functions of Matrices: Theory and Computation.

SIAM, Philadelphia, PA, USA, 2008. xx+425 pp. ISBN 978-0-898716-46-7.



Nicholas J. Higham and Srikara Pranesh.
Simulating low precision floating-point arithmetic.
SIAM J. Sci. Comput., 41(5):C585–C602, 2019.



Nick Komaroff.
Simultaneous eigenvalue lower bounds for the Lyapunov matrix equation.
IEEE Trans. Automat. Control, 33(1):126–128, 1988.



Vladimir B. Larin and Fikret A. Aliev.
Construction of square root factor for solution of the Lyapunov matrix equation.
Syst. Control Lett., 20(2):109–112, 1993.



John D. Roberts.
Linear model reduction and solution of the algebraic Riccati equation by use of the sign function.
Int. J. Control, 32(4):677–687, 1980.